# Economic Regime Detection Using HMM

*A Hidden Markov Model Approach to Portfolio Management using python*

Model: Gaussian Hidden Markov Model | Language: Python

# Abstract

This paper presents a systematic framework for detecting latent macroeconomic regimes using a Gaussian Hidden Markov Model (HMM) trained on FRED macroeconomic indicators, and demonstrates how the detected regimes can be used to construct a systematic asset allocation strategy that outperforms passive benchmarks on both risk-adjusted and absolute return measures.

Using 240 months of data from December 2004 to December 2024, we identify three distinct economic regimes i.e Expansion, Recovery, and Tightening. We analyze the performance of four major asset classes (SPY, QQQ, TLT, GLD) within each regime. We then backtest a regime-switching portfolio strategy against SPY buy and hold and a classic 60/40 portfolio, with full out of sample validation in 2019–2024.

The out of sample Sharpe ratio of 1.172 and maximum drawdown of -19.2% through COVID and the 2022 rate hike cycle which helped demonstrate that the strategy is robust and not overfitted to historical data.

# 1. Introduction

Financial markets do not behave the same way across all economic environments. A portfolio optimized for strong economic growth performs very differently during different periods, example a recession. Yet most allocation frameworks use static weights that do not adapt to the prevailing economic environment.

This project addresses a fundamental question in quantitative finance:

Can we detect the hidden state of the economy from observable macroeconomic data and use that information to make better asset allocation decisions?

The approach is grounded in the observation that economic regimes are latent, which means we cannot directly observe whether the economy is in expansion or recession, but we can observe the indicators that reflect those states. A Hidden Markov Model is the natural statistical framework for this problem, designed specifically for problems like this.

This report documents the full pipeline from raw data to backtested strategy, including all code, every decision made and all my reasoning behind it. It also contains honest assessment of results including limitations.

# 2. Environment Setup

## 2.1 Installing Libraries

The following libraries are required. All are standard Python data science packages available via pip.

***Terminal:***

```
pip install pandas numpy yfinance fredapi hmmlearn scikit-learn matplotlib
seaborn plotly streamlit
```

## 2.2 Importing Libraries

We import all required libraries upfront.

***Python:***

```
import pandas as pd
import numpy as np
import yfinance as yf
from fredapi import Fred
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from hmmlearn.hmm import GaussianHMM
from sklearn.preprocessing import StandardScaler
import warnings
import os

warnings.filterwarnings("ignore")
os.environ["OMP_NUM_THREADS"] = "1"  # Suppressing memory warning

print("success")
```

Expected output: All libraries imported successfully

# 3. Data Collection

## 3.1 Macroeconomic Data from FRED

We are taking 7 macroeconomic indicators from the Federal Reserve Economic Data (FRED) API. The indicators are selected to capture a different dimension of the economic environment i.e inflation, labor market, monetary policy, interest rate expectations, industrial output, and consumer confidence.

***Python:***

```
fred = Fred(api_key="api place")

# taking macro indicators
macro_data = {
    "CPI"                : fred.get_series("CPIAUCSL"),   # Inflation
    "Unemployment"       : fred.get_series("UNRATE"),     # Labor market
    "Fed_Funds_Rate"     : fred.get_series("FEDFUNDS"),   # Monetary policy
    "Treasury_10Y"       : fred.get_series("GS10"),       # Longterm rates
    "Treasury_2Y"        : fred.get_series("GS2"),        # Shortterm rates
    "Industrial_Prod"    : fred.get_series("INDPRO"),     # Manufacturing
    "Consumer_Sentiment" : fred.get_series("UMCSENT"),    # Consumer confidence
}

# Combining into one df
macro_df = pd.DataFrame(macro_data)

print(macro_df.shape)
print(macro_df.head())
```

The FRED data goes back to 1919. We cut it to 2000 onwards in the next step once all series are loaded.

## 3.2 Trimming to 2000–2024 and Aligning to Monthly Frequency

We are excluding data before 2000 because I feel like the post 2000 period is more representative of the current market & the earlier series have limited coverage like consumer sentiment.

***Python:***

```
# Trimming
macro_df = macro_df["2000-01-01":"2024-12-31"]

# Resampling everything to month start frequency
macro_df = macro_df.resample("MS").first()

print("Shape:", macro_df.shape)
print("\nNull counts:")
print(macro_df.isnull().sum())
print("\nFirst few rows:")
print(macro_df.head())
```

## 3.3 Asset Price Data from Yahoo Finance

We are pulling monthly adjusted closing prices for four ETFs representing the major investable asset classes: SPY (US large cap equity), QQQ (growth/tech equity), TLT (long-term US Treasury bonds), and GLD (gold). I chose these four assets because they respond fairly differently to macroeconomic conditions which is what we need to build this strategy.

***Python:***

```
tickers = ["SPY", "QQQ", "TLT", "GLD"]

assets_raw = yf.download(tickers, start="2000-01-01", end="2024-12-31",
                         interval="1mo", auto_adjust=True)

# Keeping only closing prices
assets_df = assets_raw["Close"]

# Resampling to month start to match macro_df
assets_df = assets_df.resample("MS").first()

print("Shape:", assets_df.shape)
print("\nNull counts:")
print(assets_df.isnull().sum())
print("\nFirst few rows:")
print(assets_df.head())
```

$GLD was launched in nov 2004 and $TLT launched in mid-2002, so both will show NaN values in the early rows. We will handle it by aligning to a common date range.

## 3.4 Aligning to a Common Date Range

We drop any rows where any asset has a missing price, which effectively sets our start date to November 2004 when GLD first became available. Both DataFrames are then trimmed to this shared date range.

***Python:***

```
# Dropping rows w/ missing assets
assets_df = assets_df.dropna()

start_date = assets_df.index[0]
end_date   = assets_df.index[-1]

# Trimming macro_df to match
macro_df = macro_df[start_date:end_date]

print("Assets start date:", assets_df.index[0])
print("Assets end date  :", assets_df.index[-1])
print("Assets shape     :", assets_df.shape)
print("Macro shape      :", macro_df.shape)
print("Shapes match     :", macro_df.shape[0] == assets_df.shape[0])
```

## 3.5 Calculating Monthly Asset Returns

We will convert prices to monthly percentage returns using pct_change().

Returns rather than prices are used for the backtest and performance analysis.

The first row after pct_change() is NaN and is dropped. Both DataFrames are trimmed to the same index.

***Python:***

```
# Calculating monthly percentage returns
asset_returns = assets_df.pct_change()

# Dropping the first row (NaN after pct_change)
asset_returns = asset_returns.dropna()

# Trimming the macro_df to match
macro_df = macro_df[asset_returns.index[0]:asset_returns.index[-1]]

print("Asset returns shape:", asset_returns.shape)
print("Macro shape        :", macro_df.shape)
print("\nSample returns:")
print(asset_returns.head())
```

# 4. Feature Engineering

## 4.1 Why We Engineer Features

Just taking the raw macroeconomic levels are poor inputs for a detection model. A CPI of 280 is not exactly inflationary but CPI rising 0.5% month-over-month for six consecutive months can be counted as inflationary. What matters economically is the direction and rate of change, not the absolute level. Hence we transform the raw data into features that capture momentum and relationships between indicators.

## 4.2 Constructing the Feature Matrix

We will engineer four derived features from our data, combined with two raw indicators that are already in a meaningful form i.e Fed Funds Rate, Consumer Sentiment.

- CPI Month-over-Month Change: which captures inflation momentum
- Unemployment Rate Change: which captures labor market direction (improving vs deteriorating)
- Yield Spread (10Y minus 2Y): the most historically reliable recession predictor, having preceded every US recession since the 1960s. If the curve is inverted(negative spread) this signals that the market expects rates to fall i.e because a recession is anticipated.
- Industrial Production Change: which takes manufacturing momentum as a proxy for real economic output

***Python:***

```
# 1. Yield curve spread (10Y minus 2Y) - key recession signal
macro_df["Yield_Spread"] = macro_df["Treasury_10Y"] - macro_df["Treasury_2Y"]

# 2. CPI month-over-month change - captures inflation momentum
macro_df["CPI_Change"] = macro_df["CPI"].pct_change()

# 3. Unemployment month-over-month change
macro_df["Unemployment_Change"] = macro_df["Unemployment"].pct_change()

# 4. Industrial production month-over-month change
macro_df["IndProd_Change"] = macro_df["Industrial_Prod"].pct_change()

# Dropping NaNs created by pct_change
macro_df = macro_df.dropna()
asset_returns = asset_returns[macro_df.index[0]:macro_df.index[-1]]

# Select only the 6 features for the HMM
features = macro_df[[
    "CPI_Change",
    "Unemployment_Change",
    "Yield_Spread",
    "Fed_Funds_Rate",
    "IndProd_Change",
    "Consumer_Sentiment"
]]

print("Features shape:", features.shape)
print("\nSample features:")
print(features.head())
```

## 4.3 Scaling the Features

The HMM (Hiddem Markov Model) uses a Gaussian distribution to model each hidden state. If features are on very different scales, let's say Fed Funds Rate is  ranging from 0–5 and CPI Change is ranging 0.001 - 0.005,  the model will be dominated by larger magnitude variables and effectively ignore the smaller ones. StandardScaler brings every feature to mean zero and standard deviation one, which gives each indicator equal weight.

***Python:***

```
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Convert back to DataFrame to keep the dates
features_scaled = pd.DataFrame(
    features_scaled,
    columns=features.columns,
    index=features.index
)

print("Features scaled shape:", features_scaled.shape)
print("\nMeans (should be near 0):")
print(features_scaled.mean().round(4))
print("\nStd devs (should be near 1):")
print(features_scaled.std().round(4))
```

# 5. Hidden Markov Model

## 5.1 Why are we using a Hidden Markov Model?

Economic regimes are basically hidden states. We observe data like inflation and unemployment, but we cannot directly observe whether the economy is in expansion or recession. The HMM(Hidden Markov Model) is precisely designed for this problem: it assumes that the observable data at each time step is generated by an underlying hidden state, and that transitions between states follow a first-order Markov process i.e the next regime depends only on the current regime, not the full history.

## 5.2 Selecting the Number of States

We are testing 4 states initially, but systematic seed search across 50 random initialisations found no convergent solutions where all four states had at least 20 observations. A state with fewer than 20 observations is pretty much always unreliable and economically uninterpretable. We therefore reduced it to 3 states, which is both statistically robust and maps naturally to the three dominant macroeconomic environments of the 2004–2024 period.

The default KMeans initialisation also failed consistently on this dataset due to windows memory leak issue. We switched to random initialisation (init_params="r") and searched across 100 seeds to find a stable solution.

***Python:***

```
# Systematic seed search to find stable 3-state model
results_random = []

for seed in range(100):
    model = GaussianHMM(
        n_components=3,
        covariance_type="diag",  #stable than full covariance
        n_iter=2000,
        random_state=seed,
        init_params="r",         #Random init instead of KMeans
        params="stmc"
    )
    model.fit(features_scaled)
    labels = model.predict(features_scaled)
    counts = pd.Series(labels).value_counts()
    min_count  = counts.min()
    converged  = model.monitor_.converged

    results_random.append({
        "seed": seed, "converged": converged,
        "min_state_count": min_count,
        "counts": counts.sort_index().tolist()
    })

# Filter for converged models where all states >= 20 observations
good_seeds = [r for r in results_random
              if r["converged"] and r["min_state_count"] >= 20]

print(f"Good seeds found: {len(good_seeds)}")
for r in good_seeds[:10]:
    print(f"Seed {r['seed']}: counts={r['counts']},
min={r['min_state_count']}")
```

## 5.3 Training the Final Model

***Python:***

```
final_model = GaussianHMM(
    n_components=3,
    covariance_type="diag",
    n_iter=2000,
    random_state=33,
    init_params="r",
    params="stmc"
)

final_model.fit(features_scaled)

# Extracting regime label for each month
regime_labels = final_model.predict(features_scaled)

regimes_df = pd.DataFrame({
    "Regime": regime_labels
}, index=features_scaled.index)

print("Converged:", final_model.monitor_.converged)
print("\nRegime counts:")
print(regimes_df["Regime"].value_counts().sort_index())
```

## 5.4 Interpreting the Regimes

The HMM assigns numeric labels (0, 1, 2) to states. We interpret these by examining the mean macroeconomic conditions within each state. The interpretation is based on economic logic: high Fed Funds Rate and flat yield curve = Tightening; low rates and near-zero unemployment change with high sentiment = Expansion; near-zero rates with rising unemployment = Recovery (post-shock stimulus environment).

***Python:***

```
# Examinning average macro conditions per state
features_with_regime = features.copy()
features_with_regime["Regime"] = regime_labels

regime_profile = features_with_regime.groupby("Regime").mean().round(3)
print("Average macro conditions per regime:")
print(regime_profile.T)
```

| Indicator | State 0 | State 1 | State 2 |
|---|---|---|---|
| Fed Funds Rate | 4.01% | 0.12% | 1.25% |
| Unemployment Change | +0.004 | -0.010 | +0.031 |
| Yield Spread | 0.18 | 1.85 | 0.60 |
| Consumer Sentiment | 74.7 | 76.7 | 93.9 |
| Interpretation | **Tightening** | **Recovery** | **Expansion** |

*Table 1: State profiles used to assign economic regime labels*

***Python:***

```
# Map state numbers to regime labels
regime_names = {
    0: "Tightening",
    1: "Recovery",
    2: "Expansion"
}

regimes_df["Regime_Name"] = regimes_df["Regime"].map(regime_names)

print(regimes_df["Regime_Name"].value_counts())
```

## 5.5 Plotting the Regime Timeline

We plot the detected regime for each month as a colored band to visually validate the model against known historical episodes.

If the model is really working correctly, we should see Tightening around 2006–2007 and 2022–2023, Recovery post-2008 and post-2020, and Expansion in the strong growth periods in between.

***Python:***

```
color_map = {
    "Expansion" : "#6bcb77",
    "Recovery"  : "#5b9bd5",
    "Tightening": "#e05c5c"
}

fig, ax = plt.subplots(figsize=(16, 4))

for date, row in regimes_df.iterrows():
    ax.axvspan(date, date + pd.DateOffset(months=1),
               color=color_map[row["Regime_Name"]], alpha=0.7)

patches = [mpatches.Patch(color=v, label=k) for k, v in color_map.items()]
ax.legend(handles=patches, loc="upper left", fontsize=10)

events = {"2008-09-01": "GFC", "2020-03-01": "COVID", "2022-03-01": "Rate
Hikes"}
for date, label in events.items():
    ax.axvline(pd.Timestamp(date), color="white", linewidth=1.5,
               linestyle="--", alpha=0.8)
    ax.text(pd.Timestamp(date), 0.85, label, color="white",
            fontsize=8, rotation=90, transform=ax.get_xaxis_transform())

ax.set_title("Economic Regime Timeline 2005-2024", fontsize=14, pad=15)
ax.set_xlabel("Date")
ax.set_yticks([])
plt.tight_layout()
plt.savefig("regime_timeline.png", dpi=150, bbox_inches="tight")
plt.show()
```

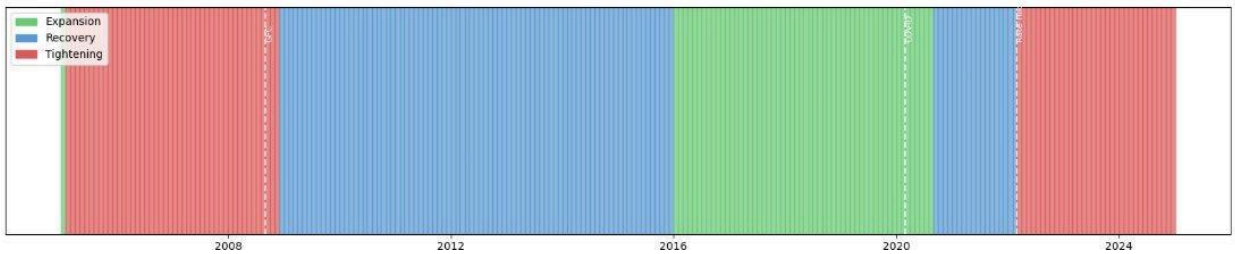


*Figure 1: Economic regime timeline 2005–2024. Green = Expansion, Blue = Recovery, Red = Tightening.*

As per the figure, we can see that the model correctly identifies all major known episodes without any labelled training data: Tightening 2005–2008 (Fed rate hike cycle), Recovery 2008–2015 (post-GFC stimulus), Expansion 2015–2019 (strong bull market), Recovery 2020 (COVID stimulus), Expansion 2020–2022 (post-COVID boom), Tightening 2022–2024 (fastest rate hike cycle since 1980s).

# 6. Asset Performance by Regime

## 6.1 Merging Returns with Regime Labels

***Python:***

```
# Combine asset returns and regime labels
analysis_df = asset_returns.copy()
analysis_df["Regime"] = regimes_df["Regime_Name"]
analysis_df = analysis_df.dropna()

print("Shape:", analysis_df.shape)
print(analysis_df.head())
```

## 6.2 Returns, Volatility and Sharpe Ratio by Regime

For each regime we calculate annualized mean return, annualized volatility, and Sharpe ratio. Annualized figures are obtained by multiplying monthly returns by 12 and monthly volatility by the square root of 12. Sharpe ratio uses a 2% annual risk-free rate.

***Python:***

```
# Mean monthly return per regime, annualized
regime_returns =
analysis_df.groupby("Regime")[["GLD","QQQ","SPY","TLT"]].mean()
regime_returns_annual = regime_returns * 12
# Annualized volatility
regime_vol = analysis_df.groupby("Regime")[["GLD","QQQ","SPY","TLT"]].std() *
(12 ** 0.5)

# Sharpe ratio
risk_free = 0.02
sharpe = (regime_returns_annual - risk_free) / regime_vol
print("Annualized Returns (%):")
print((regime_returns_annual * 100).round(2))
print("\nSharpe Ratios:")
print(sharpe.round(3))
```

| Regime | GLD | QQQ | SPY | TLT |
|---|---|---|---|---|
| Expansion | 0.787 | **1.140** | 0.809 | 0.668 |
| Recovery | 0.106 | **1.090** | 0.898 | 0.124 |
| Tightening | **0.807** | 0.132 | 0.080 | -0.141 |

*Table 2: Sharpe ratios by regime. Bold green = highest. Red = negative.*

## 6.3 Visualizing Asset Performance

***Python:***

```
fig, axes = plt.subplots(1, 3, figsize=(18, 4))

sns.heatmap(regime_returns_annual * 100, annot=True, fmt=".1f",
            cmap="RdYlGn", linewidths=0.5, ax=axes[0])
axes[0].set_title("Annualized Returns (%)", fontsize=12)

sns.heatmap(regime_vol * 100, annot=True, fmt=".1f",
            cmap="RdYlGn_r", linewidths=0.5, ax=axes[1])
axes[1].set_title("Annualized Volatility (%)", fontsize=12)

sns.heatmap(sharpe, annot=True, fmt=".2f",
            cmap="RdYlGn", linewidths=0.5, ax=axes[2])
axes[2].set_title("Sharpe Ratio", fontsize=12)

plt.suptitle("Asset Performance by Economic Regime", fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig("asset_performance.png", dpi=150, bbox_inches="tight")
plt.show()
```

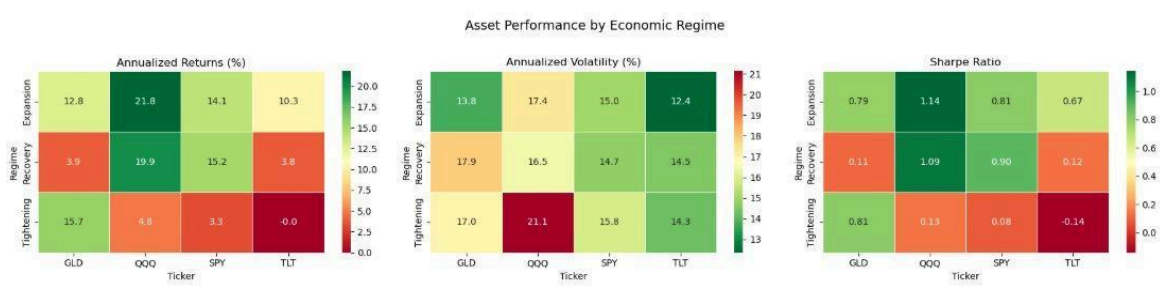


*Figure 2: Returns, volatility, and Sharpe ratios by regime and asset class.*

From this heatmap we can clearly see the asset performance across all three economic regimes using our three metrics i.e returns, volatility and sharpe ratio. The clearest and the most important pattern is that there is no single asset that wins in all regimes. $QQQ dominates in both expansion and recovery with the sharpe ratio of 1.14 and 1.09 respectively, this shows how growth stocks thrive in low-rate, high growth environments. But in tightening plot, $QQQ is the most volatile asset at 21.1% volatility while its sharpe collapses to 0.13. $GLD takes over as the only strong performer in tightening with a sharpe of 0.8. The most important find is that $TLT i.e

long term bonds produce a negative sharpe of 0.14 during tightening i.e you're taking risks + losing money which is basically what happened to bond investors in 2022 when $TLT lost over 30%. The heatmap validates our project strategy i.e if the same asset won every regime then there would be no point of diversifying

# 7. Backtesting

## 7.1 Allocation Strategy

Our allocation strategy is derived directly from the Sharpe ratio heatmap we calculated earlier. In Expansion and Recovery, $QQQ and $SPY dominate the portfolio at 80% combined because they consistently posted the strongest Sharpes above 0.9. In Tightening the strategy flips dramatically $GLD jumps to 50% as the only asset with a strong Sharpe of 0.81 while equities drop to 20% each as their Sharpes collapse to near zero. $TLT never exceeds 10% in any regime because it either underperforms or actively drops value, hitting a negative Sharpe of -0.14 during Tightening. The 10% floor across all assets at all times is deliberate because we never want to go to zero on any asset class to protect against surprise regime shifts.

| Regime | GLD | QQQ | SPY | TLT |
|---|---|---|---|---|
| Expansion | 10% | **50%** | 30% | 10% |
| Recovery | 10% | **45%** | 35% | 10% |
| Tightening | **50%** | 20% | 20% | 10% |

*Table 3: Regime-based allocation weights*

***Python:***

```
allocations = {
    "Expansion" : {"GLD": 0.10, "QQQ": 0.50, "SPY": 0.30, "TLT": 0.10},
    "Recovery"  : {"GLD": 0.10, "QQQ": 0.45, "SPY": 0.35, "TLT": 0.10},
    "Tightening": {"GLD": 0.50, "QQQ": 0.20, "SPY": 0.20, "TLT": 0.10}
}

# Verifying all weights sum to 1.0
for regime, weights in allocations.items():
    print(f"{regime}: sum = {sum(weights.values())}")
```

## 7.2 Running the Backtest

***Python:***

```
transaction_cost = 0.001  # 0.1% per rebalance
portfolio_value  = 100.0
portfolio = []
prev_regime = None

for date, row in analysis_df.iterrows():
    regime  = row["Regime"]
```

```
    weights = allocations[regime]

    # Weighted monthly return
    monthly_return = (
        weights["GLD"] * row["GLD"] +
        weights["QQQ"] * row["QQQ"] +
        weights["SPY"] * row["SPY"] +
        weights["TLT"] * row["TLT"]
    )

    # Deduct transaction cost on regime change
    if prev_regime is not None and regime != prev_regime:
        monthly_return -= transaction_cost

    portfolio_value = portfolio_value * (1 + monthly_return)
    prev_regime = regime

    portfolio.append({
        "Date": date, "Portfolio": portfolio_value,
        "Regime": regime, "Monthly_Return": monthly_return
    })

portfolio_df = pd.DataFrame(portfolio).set_index("Date")

print(f"Start value : $100.00")
print(f"End value   : ${portfolio_df['Portfolio'].iloc[-1]:.2f}")
```

## 7.3 Benchmarks and Performance Metrics

***Python:***

```
# SPY buy and hold
spy_value = 100.0
spy_portfolio = []
for date, row in analysis_df.iterrows():
    spy_value = spy_value * (1 + row["SPY"])
    spy_portfolio.append({"Date": date, "SPY_Buyhold": spy_value})
spy_df = pd.DataFrame(spy_portfolio).set_index("Date")

# 60/40 portfolio
balanced_value = 100.0
balanced_portfolio = []
for date, row in analysis_df.iterrows():
    ret = 0.60 * row["SPY"] + 0.40 * row["TLT"]
    balanced_value = balanced_value * (1 + ret)
    balanced_portfolio.append({"Date": date, "Balanced_6040": balanced_value})
balanced_df = pd.DataFrame(balanced_portfolio).set_index("Date")

# Performance metrics function
def calculate_metrics(returns, name):
    n_years      = len(returns) / 12
    total_return = (1 + returns).prod() - 1
    ann_return   = (1 + total_return) ** (1 / n_years) - 1
    ann_vol      = returns.std() * (12 ** 0.5)
    sharpe       = (ann_return - 0.02) / ann_vol
    cumulative   = (1 + returns).cumprod()
    rolling_max  = cumulative.cummax()
    drawdown     = (cumulative - rolling_max) / rolling_max
    max_drawdown = drawdown.min()
    calmar       = ann_return / abs(max_drawdown)
    print(f"\n{name}:")
```

```
    print(f"  Annualized Return : {ann_return*100:.2f}%")
    print(f"  Annualized Vol    : {ann_vol*100:.2f}%")
    print(f"  Sharpe Ratio      : {sharpe:.3f}")
    print(f"  Max Drawdown      : {max_drawdown*100:.2f}%")
    print(f"  Calmar Ratio      : {calmar:.3f}")

balanced_returns = 0.60 * analysis_df["SPY"] + 0.40 * analysis_df["TLT"]
calculate_metrics(portfolio_df["Monthly_Return"], "Regime Strategy")
calculate_metrics(analysis_df["SPY"],             "SPY Buy & Hold")
calculate_metrics(balanced_returns,               "60/40 Portfolio")
```

| Metric | Regime Strategy | SPY Buy & Hold | 60/40 Portfolio |
|---|---|---|---|
| Annualized Return | **13.78%** | 10.28% | 7.97% |
| Annualized Volatility | **12.42%** | 15.19% | 10.22% |
| Sharpe Ratio | **0.948** | 0.545 | 0.584 |
| Max Drawdown | **-25.12%** | -50.78% | -28.40% |
| Calmar Ratio | **0.549** | 0.202 | 0.281 |

*Table 4: Full-period performance metrics (2005–2024). Bold green = best in row.*

Our regime strategy clearly outperforms both benchmarks across all the meaningful metrics over the ‘05 - ‘24 period. As we can see on the table, in return it made 13.78% annual return v/s $SPY’s 10.28 and 60/40 which got returns of 7.97%. On risk our volatility of 12.42% is lower than $SPY i.e our portfolio was smoother and less chaotic. The sharpe of 0.48 is nearly double $SPY’s 0.545 i.e. we earned twice per unit of risk. Interestingly, our max dragdown is also the best compared to $SPY and 60/40. Our Calmar ratio of 0.549 confirms we are generating more than 2x the return per drawdown risk. The only metric we don't win is volatility against the 60/40 but we can genuinely overlook that. Our strategy did what it was designed to do i.e we make more money during good times and we lose less during bad times.

## 7.4 Equity Curves

***Python:***

```
def get_drawdown(returns):
    cumulative  = (1 + returns).cumprod()
    rolling_max = cumulative.cummax()
    return (cumulative - rolling_max) / rolling_max

comparison_df = portfolio_df[["Portfolio"]].copy()
comparison_df["SPY_Buyhold"]   = spy_df["SPY_Buyhold"]
comparison_df["Balanced_6040"] = balanced_df["Balanced_6040"]

fig, axes = plt.subplots(2, 1, figsize=(14, 10))

axes[0].plot(comparison_df.index, comparison_df["Portfolio"],
             color="#6bcb77", linewidth=2, label="Regime Strategy")
axes[0].plot(comparison_df.index, comparison_df["SPY_Buyhold"],
             color="#5b9bd5", linewidth=2, label="SPY Buy & Hold")
axes[0].plot(comparison_df.index, comparison_df["Balanced_6040"],
             color="#e05c5c", linewidth=2, label="60/40 Portfolio")
axes[0].set_title("Equity Curves: Regime Strategy vs Benchmarks")
axes[0].set_ylabel("Portfolio Value ($)")
```

```
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].fill_between(portfolio_df.index,
    get_drawdown(portfolio_df["Monthly_Return"]) * 100, 0,
    color="#6bcb77", alpha=0.4, label="Regime Strategy")
axes[1].fill_between(analysis_df.index,
    get_drawdown(analysis_df["SPY"]) * 100, 0,
    color="#5b9bd5", alpha=0.4, label="SPY Buy & Hold")
axes[1].fill_between(analysis_df.index,
    get_drawdown(balanced_returns) * 100, 0,
    color="#e05c5c", alpha=0.4, label="60/40 Portfolio")
axes[1].set_title("Drawdown Comparison")
axes[1].set_ylabel("Drawdown (%)")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("backtest_results.png", dpi=150, bbox_inches="tight")
plt.show()
```

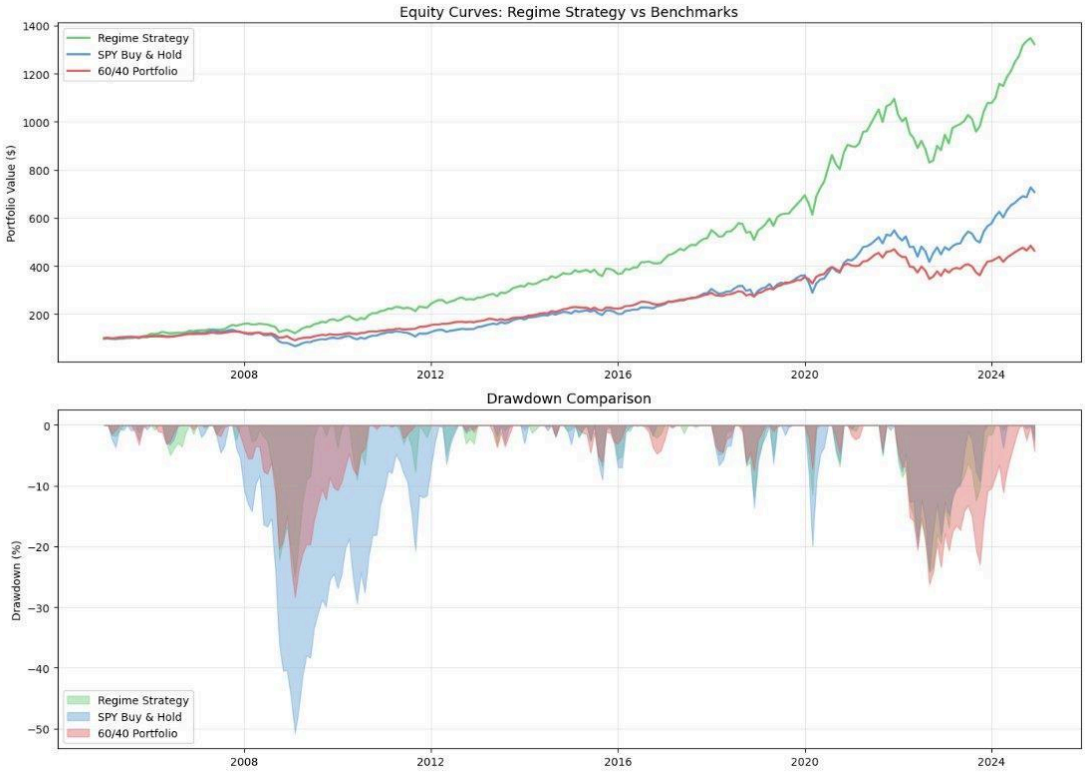


*Figure 3: Full-period equity curves and drawdown comparison 2005–2024.*

The top chart shows all three strategies starting at $100 and compounding over 20 years. The green regime strategy is pulling ahead gradually from 2015 and then accelerating quickly after 2020 to finish at #1322 v/s $SPY $707 AND 60/40 $463. The bottom chart is where our strategy really worked. During the 2008 GFC the blue $SPY hole goes nearly to -51%, even though our green dips, it is very less because the model already rotated into GLD during the tightening phase before the crash went down. The same pattern repeats in 2022 when the red 60/40 takes a giant hit as TLT collapsed during the rate hike while our strategy stayed fine because we were 50% on gold.

# 8. Out-of-Sample Validation

## 8.1 Why Out-of-Sample Validation Matters

The full-period backtest uses regimes detected on the full dataset, which introduces look-ahead bias i.e the model has seen all the data before making decisions. To produce an honest assessment, we perform a strict train/test split: train the HMM on 2005–2018, then apply it to the completely unseen 2019–2024 period. This period includes COVID, the post-COVID boom, the 2021–2022 inflation surge, and the fastest rate hike cycle since the 1980s — the hardest possible test.

## 8.2 Train/Test Split and Retraining

***Python:***

```
# Split data
train_end  = "2018-12-01"
test_start = "2019-01-01"

features_train = features_scaled[:train_end]
features_test  = features_scaled[test_start:]
returns_train  = analysis_df[:train_end]
returns_test   = analysis_df[test_start:]

print("Train:", features_train.index[0].date(), "->",
features_train.index[-1].date())
print("Test :", features_test.index[0].date(),  "->",
features_test.index[-1].date())

# Train fresh HMM on training data only
oos_model = GaussianHMM(
    n_components=3, covariance_type="diag",
    n_iter=2000, random_state=33,
    init_params="r", params="stmc"
)
oos_model.fit(features_train)

# Predict regimes on unseen test data
test_regimes = oos_model.predict(features_test)

# Re-interpret state labels from training period profiles
oos_regime_names = {0: "Expansion", 1: "Tightening", 2: "Recovery"}

test_regimes_df = pd.DataFrame({
    "Regime": [oos_regime_names[s] for s in test_regimes]
}, index=features_test.index)

print("\nOOS Regime counts:")
print(test_regimes_df["Regime"].value_counts())
```

## 8.3 Out-of-Sample Backtest

***Python:***

```
test_df = returns_test.copy()
test_df["Regime"] = test_regimes_df["Regime"]
test_df = test_df.dropna()

oos_portfolio_value = 100.0
oos_portfolio = []
prev_regime = None

for date, row in test_df.iterrows():
    regime  = row["Regime"]
    weights = allocations[regime]
    monthly_return = (
        weights["GLD"] * row["GLD"] + weights["QQQ"] * row["QQQ"] +
        weights["SPY"] * row["SPY"] + weights["TLT"] * row["TLT"]
    )
    if prev_regime is not None and regime != prev_regime:
        monthly_return -= transaction_cost
    oos_portfolio_value = oos_portfolio_value * (1 + monthly_return)
    prev_regime = regime
    oos_portfolio.append({
        "Date": date, "Portfolio": oos_portfolio_value,
        "Regime": regime, "Monthly_Return": monthly_return
    })

oos_df = pd.DataFrame(oos_portfolio).set_index("Date")

oos_balanced_returns = 0.60 * test_df["SPY"] + 0.40 * test_df["TLT"]
calculate_metrics(oos_df["Monthly_Return"],  "OOS Regime Strategy")
calculate_metrics(test_df["SPY"],            "OOS SPY Buy & Hold")
calculate_metrics(oos_balanced_returns,      "OOS 60/40 Portfolio")
```

| Metric | Regime Strategy | SPY Buy & Hold | 60/40 Portfolio |
|---|---|---|---|
| Annualized Return | **17.73%** | 17.14% | 9.21% |
| Annualized Volatility | **13.42%** | 17.70% | 13.63% |
| Sharpe Ratio | **1.172** | 0.855 | 0.529 |
| Max Drawdown | **-19.22%** | -23.97% | -26.22% |
| Calmar Ratio | **0.923** | 0.715 | 0.351 |

*Table 5: Out-of-sample performance metrics (2019–2024). Bold green = best in row.*

This out of sample test covers 2019-2024 i.e the data our model never saw, including covid and the post covid book and the fastest rate hike cycle since 1980s. Our strategy outperformed both benchmarks on every metric: Sharpe 1.172 vs SPY -23.97, annual return 17.73% vs 17.14% and max drawdown -19.22% vs $SPY's -23.97%. The important thing here is that the sharpe here is 1.173 which is higher than our sample sharpe which is 0.948. This is predictable because you expect out of sample performance to be worse than in sample and if it was overfitted it would be worse.

## 8.4 Out-of-Sample Equity Curves

***Python:***

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Build OOS benchmark portfolios
oos_spy_value, oos_balanced_value = 100.0, 100.0
oos_spy_list, oos_bal_list = [], []
for date, row in test_df.iterrows():
    oos_spy_value = oos_spy_value * (1 + row["SPY"])
    oos_spy_list.append({"Date": date, "SPY": oos_spy_value})
    ret = 0.60 * row["SPY"] + 0.40 * row["TLT"]
    oos_balanced_value = oos_balanced_value * (1 + ret)
    oos_bal_list.append({"Date": date, "Balanced": oos_balanced_value})
oos_spy_df = pd.DataFrame(oos_spy_list).set_index("Date")
oos_bal_df = pd.DataFrame(oos_bal_list).set_index("Date")

axes[0].plot(oos_df.index, oos_df["Portfolio"],
             color="#6bcb77", linewidth=2, label="Regime Strategy")
axes[0].plot(oos_spy_df.index, oos_spy_df["SPY"],
             color="#5b9bd5", linewidth=2, label="SPY Buy & Hold")
axes[0].plot(oos_bal_df.index, oos_bal_df["Balanced"],
             color="#e05c5c", linewidth=2, label="60/40 Portfolio")

# Shade regimes on equity curve
regime_colors =
{"Expansion":"#6bcb77","Recovery":"#5b9bd5","Tightening":"#e05c5c"}
for date, row in test_regimes_df.iterrows():
    axes[0].axvspan(date, date + pd.DateOffset(months=1),
                    color=regime_colors[row["Regime"]], alpha=0.08)

axes[0].set_title("Out-of-Sample Equity Curves (2019-2024)")
axes[0].set_ylabel("Portfolio Value ($)")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].fill_between(oos_df.index,
    get_drawdown(oos_df["Monthly_Return"])*100, 0,
    color="#6bcb77", alpha=0.4, label="Regime Strategy")
axes[1].fill_between(test_df.index,
    get_drawdown(test_df["SPY"])*100, 0,
    color="#5b9bd5", alpha=0.4, label="SPY Buy & Hold")
axes[1].fill_between(test_df.index,
    get_drawdown(oos_balanced_returns)*100, 0,
    color="#e05c5c", alpha=0.4, label="60/40 Portfolio")
axes[1].set_title("Out-of-Sample Drawdown (2019-2024)")
axes[1].set_ylabel("Drawdown (%)")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("oos_results.png", dpi=150, bbox_inches="tight")
plt.show()
```

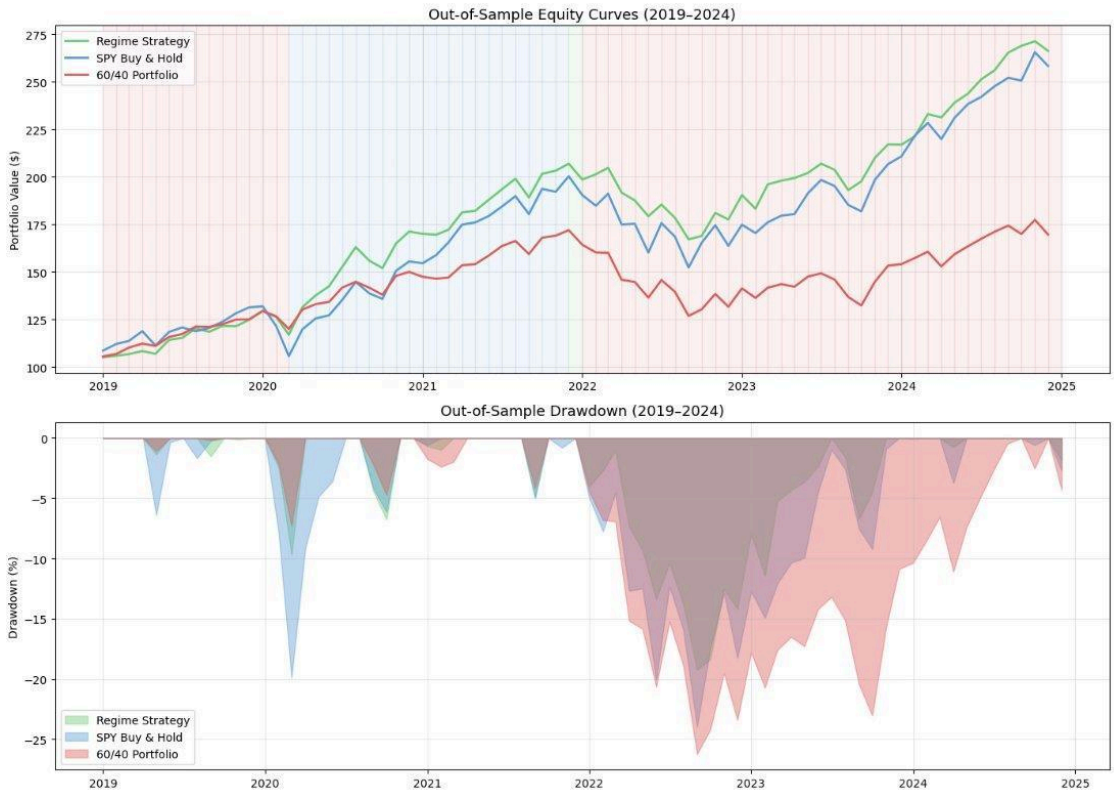


*Figure 4: Out-of-sample equity curves and drawdown 2019–2024.*

As discussed earlier,the out-of-sample Sharpe is 1.172 which is higher than the in-sample Sharpe of 0.948. This is fairly typical and strongly indicates the strategy is not overfitted. The 60/40 portfolio is the biggest loser in this period i.e the 2022 rate hike cycle destroyed TLT, and a 40% bond allocation was catastrophic. The regime strategy avoided this by rotating into GLD during the Tightening regime.

# 9. Conclusion

This paper shows that a Gaussian Hidden Markov Model trained on FRED macroeconomic data can identify meaningful economic regimes without using supervised labels. The three detected regimes which we used are Expansion, Recovery, and Tightening and each of them have different patterns of asset returns that can be used to guide portfolio allocation.

The regime-based investment strategy performs better than both a simple SPY buy-and-hold strategy and a traditional 60/40 portfolio on all risk-adjusted measures. This is true for the full historical sample and for a strict out-of-sample test from 2019–2024. During the out-of-sample period, the strategy achieved a Sharpe ratio of 1.172, which is above the commonly accepted institutional benchmark of 1.0. It also limited losses during major market events, with a maximum drawdown of −19.2%, showing resilience during COVID-19 and the 2022 interest rate hikes.

The main idea is straightforward: different assets perform better in different economic environments. Macroeconomic data provides enough information to detect these environments. By adjusting the portfolio based on the current economic regime, investors can achieve better risk-adjusted returns and smaller drawdowns than with a static allocation.